

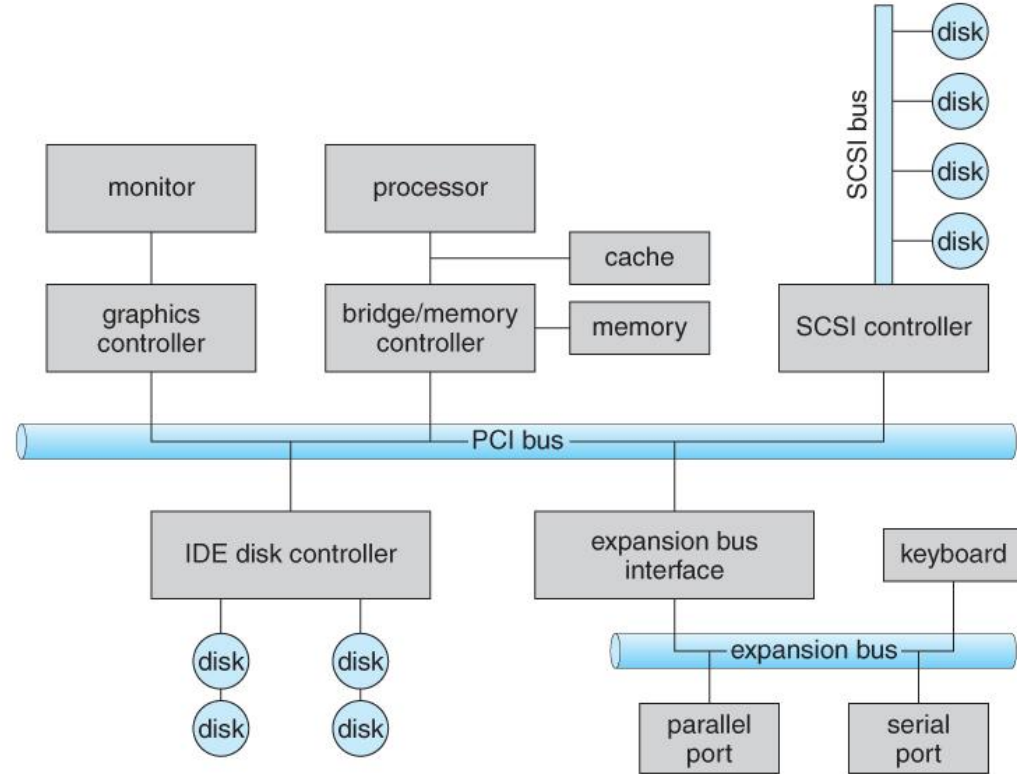
# I/O Systems-I

Dr. Jit Mukherjee



# Overview and Hardware

- I/O devices vary widely in function and speed -> I/O subsystem in kernel
- Two conflicting trends:
  - Making it easier to add newly developed devices to existing systems
  - The development of new types of devices -> difficult for existing interfaces.
- storage, communications, user-interface, and other
- Communicates -> Wire / Wireless
- Connects -> Serial, Parallel Port
- Common set of wires connecting multiple devices -> *bus*
  - Daisy Chain - A->B->C->Port (Computer)
  - PCI Bus - Processor Memory Subsystem to fast devices
  - Expansion Bus - Relatively slow devices
  - SCSI Bus - SCSI Devices, SCSI Controller



# Hardware

- Controller -> Collection of electronics -> operates port / bus / device
  - SCSI Controller is complex not as simple as serial port controller
  - Implements as a separate circuit board -> host adapter -> a processor, microcode, memory
- Controller has one or more registers for data and control signals. Processor communicates -> reading, writing bit patterns in these registers
  - A device controller supports memory mapped I/O -> Mapped to address space of processor
  - Some system uses both Controller and Memory mapped I/O
- Registers may be one to four bytes in size, and may typically include:
  - The *data-in register* is read by the host to get input from the device.
  - The *data-out register* is written by the host to send output.
  - The *status register* has bits read by the host to ascertain the status of the device, such as idle, ready for input, busy, error, transaction complete, etc.
  - The *control register* has bits written by the host to issue commands or to change settings of the device such as parity checking, word length, or full- versus half-duplex operation.
- Few Controllers have FIFO chips -> hold several bytes of input / output data

# Hardware

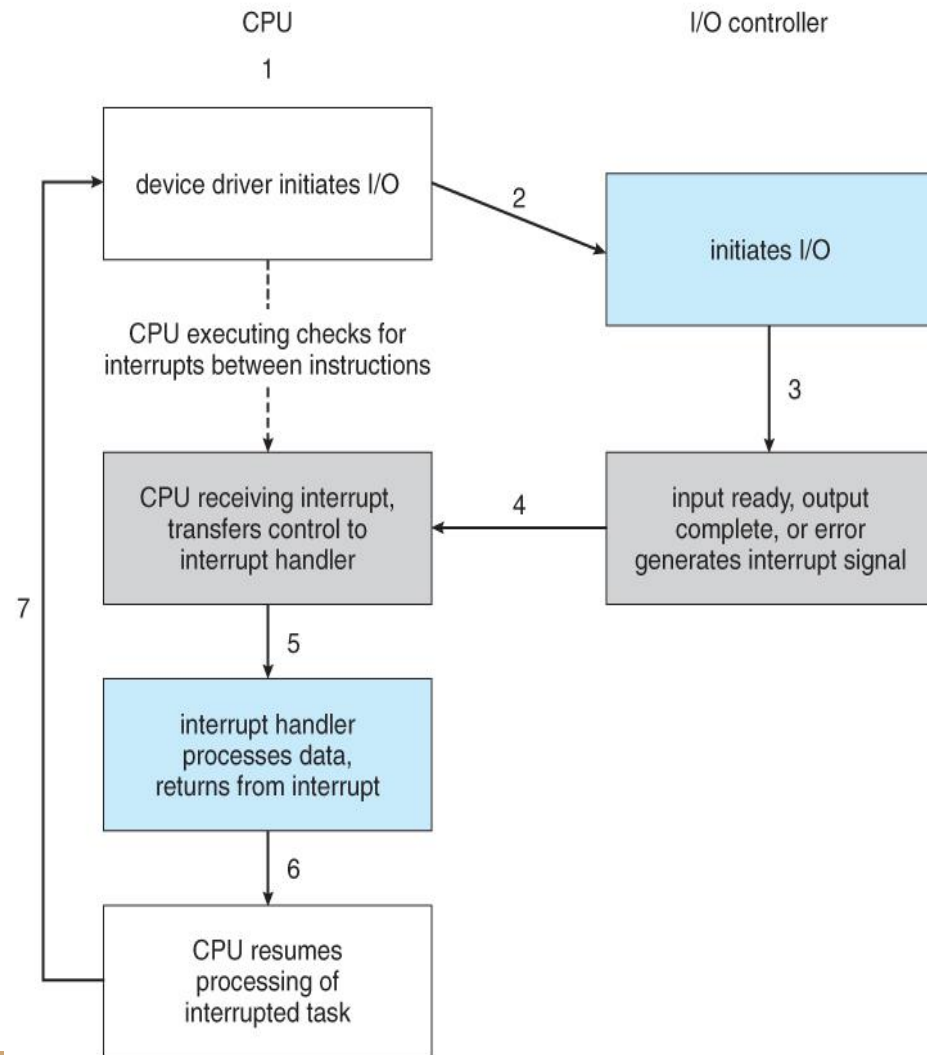
| I/O address range (hexadecimal) | device                    |
|---------------------------------|---------------------------|
| 000–00F                         | DMA controller            |
| 020–021                         | interrupt controller      |
| 040–043                         | timer                     |
| 200–20F                         | game controller           |
| 2F8–2FF                         | serial port (secondary)   |
| 320–32F                         | hard-disk controller      |
| 378–37F                         | parallel port             |
| 3D0–3DF                         | graphics controller       |
| 3F0–3F7                         | diskette-drive controller |
| 3F8–3FF                         | serial port (primary)     |

# Polling

- The complete protocol for interaction between host and controller intricate
  - Basic handshaking is simple
- 2 bits for producer consumer relationship between controller and host
  - Controller-> busy bit in the status register -> set (1) working or busy, free otherwise
  - Host signals it's wish using command-ready bit -> common register
  - Sets command ready bit when a command is available for controller to execute
  - Host writes through port coordinate with controller using handshaking
- The host repeatedly checks the *busy bit* on the device until it becomes clear -> **Polling**
- The host writes a byte of data into the data-out register, and sets the *write bit*
- The host sets the *command ready bit* in the command register to notify the device of the pending command.
- When the device controller sees the command-ready bit set, it first sets the busy bit.
- Then the device controller reads the command register, sees the write bit set, reads the byte of data from the data-out register, and outputs the byte of data.
- The device controller then clears the *error bit* in the status register, the command-ready bit, and finally clears the busy bit, signaling the completion of the operation.
- Polling can be very fast and efficient, if both the device and the controller are fast and if there is significant data to transfer. Inefficient, if the host must wait a long time in the busy loop waiting for the device, or if frequent checks need to be made for data that is infrequently there.

# Interrupts

- CPU hardware has a wire called **interrupt-request line**
  - CPU senses after every instruction -> Controller asserted a signal on the line -> Save state
- The interrupt handler determines the cause of the interrupt, do necessary
- In Modern OS -> Ability to defer interrupt handling during critical section
  - Efficient way to dispatch to the proper interrupt handler without first polling all the devices
  - Multilevel interrupts
- CPU two interrupt lines -> maskable and nonmaskable
- Accepts an address, a number, that selects a specific routine - Interrupt Vector -> Memory address for specialized interrupt handlers -> !search all source



# Interrupts Contd.

- **Interrupt Chaining** -> each element in the interrupt vector points to the head of a list of interrupt handler
  - Handler on the list call one by one
  - Huge interrupt table
- **Use of Interrupts**
  - Interrupt priority level -> preemption
  - Boot time, interrupt vector -> Checks hardware buses -> installed devices
  - I/O -> Device when ready to service
  - Handles wide variety of exceptions
  - Virtual Memory Paging. Page Fault
  - System Calls; Software interrupts / traps
  - Flow of control within the kernel

| vector number | description                            |
|---------------|----------------------------------------|
| 0             | divide error                           |
| 1             | debug exception                        |
| 2             | null interrupt                         |
| 3             | breakpoint                             |
| 4             | INTO-detected overflow                 |
| 5             | bound range exception                  |
| 6             | invalid opcode                         |
| 7             | device not available                   |
| 8             | double fault                           |
| 9             | coprocessor segment overrun (reserved) |
| 10            | invalid task state segment             |
| 11            | segment not present                    |
| 12            | stack fault                            |
| 13            | general protection                     |
| 14            | page fault                             |
| 15            | (Intel reserved, do not use)           |
| 16            | floating-point error                   |
| 17            | alignment check                        |
| 18            | machine check                          |
| 19-31         | (Intel reserved, do not use)           |
| 32-255        | maskable interrupts                    |

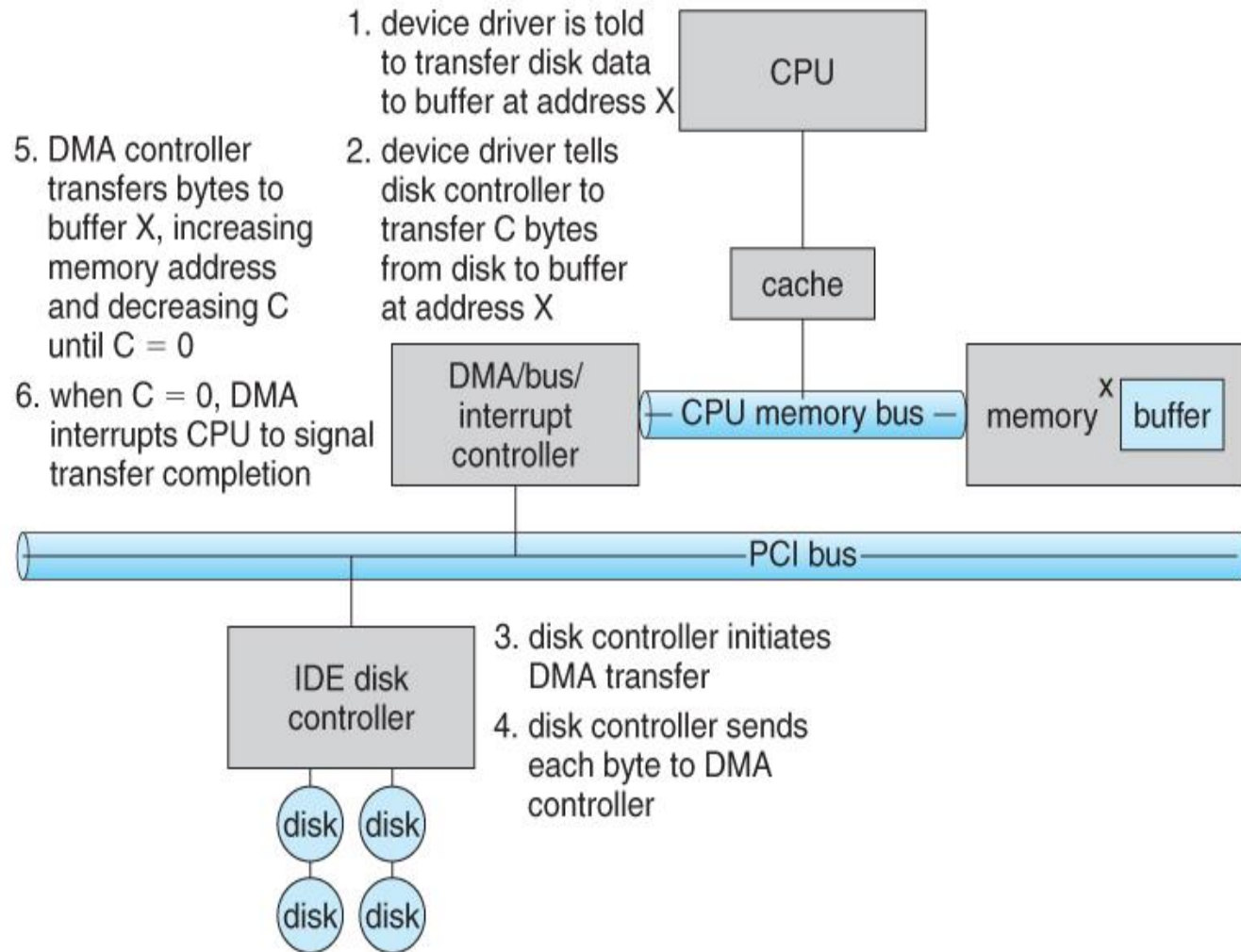
# Direct Memory Access (DMA)

- Programmed I/O (PIO) - General purpose processor to watch status bit and to free data into control register one byte at a time
  - Avoid burdening the main CPU -> Special purpose processor -> Direct Memory Access
- Host writes DMA command block into memory -> pointer to the source of a transfer -> pointer to destination -> count of number of bytes
- CPU writes the address of the command block to DMA controller -> DMA proceeds to operate the memory bus directly without the help of CPU
  - Bus-mastering I/O board contains high speed DMA hardware
- Handshaking between DMA controller and device controller -> DMA-request and DMA-acknowledge
  - Device Controller places a signal on DMA-request wire when a data is available to transfer
  - DMA Controller seizes memory bus -> Places the desired address on memory address wire
  - Place a signal to DMA-acknowledge wire.
  - DEvice controller receives DMA-acknowledge -> Transfer -> Remove DMA-request



# DMA

- After finishing DMA interrupts the CPU
- When DMA control memory bus -> CPU can not access main memory -> **Cycle stealing**
- Improves total system performance.
- **Direct Virtual Memory Access** (DVMA) - using virtual address
- Security and Protection issues



# I/O Hardware Summary

- A bus
- A controller
- An I/O port and its registers
- Handshaking relationship between host and device controller
- Execution of handshaking in a polling loop or via interrupts
- Offloading by DMA controller for large transfer